

目 录

其它	2
字符/数字转换	2
进制转换	3
精度/溢出问题	5
输入输出优化	8
交互题	10
一些语法/标准	11
重载运算符	11
Lambda表达式	13
预编译头文件	16
gcc标准	16
__int128	16
__cplusplus	18
文件读写	18
可变模版参数	19
时间复杂度	19
握手定理	22
分块打表	22
随机数生成	23
对拍	24
外部链接	27
.vimrc	28
OI	28

其它

字符/数字转换

ASCII码范围为0~127，注意不要越界，否则会乱码

字符串 <=> 数字

```
1 //字符串->数字
2 //atoi整型   atol长整型   atoll长长整型   atof浮点型           默认参数为const char*类型
3 //stoi类似,默认参数为const string*类型,且超出int范围会报错
4
5 string s1 = "123456";
6 int n = stoi(s1);           //string类型字符串
7 int n1 = atoi(s1.c_str()); //c_str() 函数可以将 const string* 类型 转化为 const
                             char* 类型
8
9 char s2[] = "123456";
10 int n2 = atoi(s2);          //char[]类型字符串
```

```
1 //数字->字符串
2 int n = 541;
3 string a = to_string(n);//#include<string>
```

```
1 //数字<=>字符串
2 //使用stringstream 输入流
3 stringstream ss;
4 int n = 123456;
5 string str;
6 ss << n;
7 ss >> str;
```

字符c <=> 数字n

```
1 //字符c 转 数字n
2 n = c - '0' ;    //或者 -48
```

```
1 //数字n 转 字符c
2 c = n + '0';
```

大小写转换

```
1 | s[i] ^= ' '; //大小写互转  空格 ' ' 为char(32)
```

```
1 | s[i] = tolower(s[i]); //转小写  s[i] |= ' '
2 | s[i] = toupper(s[i]); //转大写  s[i] &= ~' '
```

进制转换

stoi k进制转十进制

- `stoi(字符串, 0, k进制)`：将一串k进制的字符串转换为 int 型数字。
- `stoll`, `stoull`, `stod`, `stold` 同理。

```
1 | //s为0~9 + 'a'~'z'
2 | #include <iostream>
3 | using namespace std;
4 | int main(){
5 |     string s = "100";
6 |     int k = 16;
7 |     int n = stoi(s,0,k); //将k进制的字符串s转为十进制的整型n
8 |     cout << n << endl;
9 | }
```

```
1 | //s为数字
2 | ll calc(vector<int> s,int k){
3 |     ll ans = 0;
4 |     ll base = 1;
5 |     for(int i = s.size()-1; i >= 0; i--){
6 |         ans += s[i]*base;
7 |         base *= k;
8 |     }
9 |     return ans;
10 | }
```

十进制转k进制

建议直接手写进制转换函数

```
1 | //n是待转换的十进制数，m是待转换成的进制数
2 | //以数字+字母表示
3 | string intToA(int n,int k){
4 |     string ans="";
5 |     do{ //使用do循环防止n为0的情况
6 |         int t = n %k;
7 |         if(t>=0&&t<=9) ans+=(t+'0');
```

```

8         else ans+=(t-10+'a');
9         n/=k;
10    }while(n);
11    reverse(ans.begin(),ans.end());
12    return ans;
13 }
14
15 //以数字表示
16 vector<int> calc(int n , int m){
17     vector<int>ans;
18     do{
19         ans.push_back(n%m);
20         n /= m;
21     }while(n);
22     reverse(ans.begin(),ans.end());
23     return ans;
24 }

```

```

1 //sprintf十进制转8/16进制
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     int n; cin >> n;
7     char s[100];
8
9     //x/X  -16进制(大小写)
10    //o      -8进制
11    //d      -10进制
12    sprintf(s, "%x", n);
13    cout << s << endl;
14
15    return 0;
16 }

```

itoa

【注意】 不建议使用

itoa并不是一个标准的C函数，它是Windows特有的，如果要写跨平台的程序，需要用sprintf。

【函数原型】 `char *itoa(int value, char *string, int radix);`

【参数说明】

value: 要转换的数据。

string: 目标字符串的地址。

radix: 转换后的进制数，可以是10进制、16进制等，范围必须在 2-36。

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int n;cin >> n;
5      char str[100];
6      _itoa(n, str,36 );//例：10进制转36进制存于字符串str中
7      cout << str;
8      return 0;
9  }

```

精度/溢出问题

[double等浮点数比较问题，eps double eps 值-CSDN博客](#)

```

1  double n = 3.8;
2  double ans = n - int(n);
3  ans*=10;
4  int ans1 =int(ans);
5  cout << ans1 << endl;//预期为8，输出结果却是7

```

```

1  #define eps = 1e-8
2  double n = 3.8;
3  double ans = n - int(n);
4  ans*=10;
5  int ans1 =int(ans + eps);//对于负数的情况，只需要把ans + eps改为ans - eps即可
6  cout << ans1 << endl;//正确输出8

```

现在考虑一种情况,题目要求输出保留两位小数。有个case的正确答案的精确值是0.005,按理应该输出0.01,但你的结果可能是0.005000000001(恭喜)，也有可能是0.004999999999(悲剧),如果按照printf("%.2lf", a)输出，那你的遭遇将和括号里的字相同。

解决办法是，如果a为正，则输出a+eps，否则输出a-eps

两个浮点数之间的比较，eps缩写自epsilon，表示一个小量，但这个量又要确保远大于浮点运算结果的不确定量。eps最常见的取值是1e-8左右。引入eps后，我们判断两浮点数a、b相等的方式如下：

定义三出口函数如下: int sgn(double a){return a < -eps ? -1 : a < eps ? 0 : 1;}

则各种判断大小的运算都应做如下修正：

传统意义	修正写法1	修正写法2
------	-------	-------

传统意义	修正写法1	修正写法2
$a == b$	$\text{sgn}(a - b) == 0$	$\text{fabs}(a - b) < \text{eps}$
$a != b$	$\text{sgn}(a - b) != 0$	$\text{fabs}(a - b) > \text{eps}$
$a < b$	$\text{sgn}(a - b) < 0$	$a - b < -\text{eps}$
$a \leq b$	$\text{sgn}(a - b) \leq 0$	$a - b < \text{eps}$
$a > b$	$\text{sgn}(a - b) > 0$	$a - b > \text{eps}$
$a \geq b$	$\text{sgn}(a - b) \geq 0$	$a - b > -\text{eps}$

除法之间比较尽量转换为乘法之间比较，如 $\frac{a}{b} < \frac{c}{d}$ 判断改为 $a * d < b * c$

或者利用乘法逆元判断

```

1 //https://ac.nowcoder.com/acm/contest/93218/D
2 //求平面上有多少个不同的直线
3 #include <bits/stdc++.h>
4 int T = 1; using ll = long long;
5 const int mod = 1e9+7;
6 using namespace std;
7 const int N = 1003;
8 int n;
9 ll x[N],y[N];
10
11 ll qmi(ll a,ll b,ll p){
12     ll ans = 1;
13     while(b){
14         if(b&1) ans = ans*a%p;
15         b>>=1;
16         a = a*a%p;
17     }
18     return ans%p;
19 }
20
21 pair<ll,ll> uuz(ll x1,ll y1,ll x2,ll y2){
22     ll k;
23     if(y1 == y2) k = 1e18;//特判斜率不存在的情况
24     else if(x1 == x2) k = 0;
25     else k = (x1-x2)*qmi(y2-y1,mod-2,mod)%mod;//除法转乘法逆元
26     ll b = (y2*y2-y1*y1+x2*x2-x1*x1)*qmi(2*(y2-y1),mod-2,mod)%mod;
27     return {k,b};
28 }
29
30 void sol(){
31     cin >> n;

```

```

32     for(int i = 1;i <= n;i++){cin >> x[i];}
33     for(int i = 1;i <= n;i++){cin >> y[i];}
34
35     map<pair<ll,ll>,int>mp;
36
37     for(int i = 1;i <= n;i++){
38         for(int j = i+1;j <= n;j++){
39             auto [k,b] = uuz(x[i],y[i],x[j],y[j]);
40             if(k == 1e18) mp[{k,x[i]+x[j]}]++;
41             else mp[{k,b}]++;
42         }
43     }
44     cout << mp.size() << '\n';
45 }
46
47 int main() {
48     cin >> T;
49     while(T--){ sol(); }
50 }

```

sqrt

```

1  int ans1 = 0,ans2 = 0,ans3 = 0;
2  for(int i = 1;i <= 100;i++){
3      if(i == sqrt(i)*sqrt(i)) ans1++;//53
4      if(i - sqrt(i)*sqrt(i) < 1e-9) ans2++;//100
5      if(i == (int)sqrt(i)*sqrt(i)) ans3++;//10
6  }

```

sqrtl返回值为long double类型，精度更高。[Yet Another Simple Math Problem - Problem - QQJ.ac](#)

pow/ceil/floor/round参数类型和返回值类型均为浮点型，可能会导致输出与预期不符而wrong answer

```

1  int a = 1234;
2  cout << a*a << endl;//1522756
3  cout << pow(a, 2) << endl;//1.52276e+06

```

可以用int n = pow(a,2)类型转换再输出n

有符号整型 - 无符号整型

```

1  vector<int>v(3);
2  int n = 2;
3  while(n - v.size() > 0){
4      //预期2-3 = -1
5      //实际却是4294967295导致死循环
6      //应改写为while(n > s.size()) 或 while(n > (int)v.size())
7  }

```

输入输出优化

使用cin/cout会比scanf/printf慢

```

1  //关闭cstdio和iostream的同步后,cin/cout不要与printf/scanf/puts混用,也不要再使用endl,否则
    会造成输入输出混乱
2  ios::sync_with_stdio(false);
3  cin.tie(0);
4  cout.tie(0);

```

```

1  //endl每次会清空缓冲区,效率比'\n'慢很多
2  #define endl '\n'

```

```

1  //快速读写  不要与ios::sync_with_stdio(false)同时使用
2  inline int read(){
3      int x = 0,f = 1;
4      char ch = getchar();//linux可以用更快的getchar_unlocked()
5      while(ch < '0' || ch > '9'){//跳过空格回车等其他字符+判断正负
6          if(ch == '-') f = -1;
7          ch = getchar();
8      }
9      while(ch >= '0' && ch <= '9'){//读取数字
10         x = x * 10 + ch - '0';
11         ch = getchar();
12     }
13     return x * f;
14 }
15
16 void write(int x) {
17     if (x < 0) putchar('-'), x = ~x + 1;
18     if (x > 9) write(x / 10);
19     putchar(x % 10 + '0');
20 }

```

```
1 //O2优化, 程序开头加入
2 #pragma GCC optimize(2)
```

```
1 //火车头?
2 #pragma GCC optimize(3)
3 #pragma GCC target("avx")
4 #pragma GCC optimize("Ofast")
5 #pragma GCC optimize("inline")
6 #pragma GCC optimize("-fgcse")
7 #pragma GCC optimize("-fgcse-lm")
8 #pragma GCC optimize("-fipa-sra")
9 #pragma GCC optimize("-ftree-pre")
10 #pragma GCC optimize("-ftree-vrp")
11 #pragma GCC optimize("-fpeephole2")
12 #pragma GCC optimize("-ffast-math")
13 #pragma GCC optimize("-fsched-spec")
14 #pragma GCC optimize("unroll-loops")
15 #pragma GCC optimize("-falign-jumps")
16 #pragma GCC optimize("-falign-loops")
17 #pragma GCC optimize("-falign-labels")
18 #pragma GCC optimize("-fdevirtualize")
19 #pragma GCC optimize("-fcaller-saves")
20 #pragma GCC optimize("-fcrossjumping")
21 #pragma GCC optimize("-fthread-jumps")
22 #pragma GCC optimize("-funroll-loops")
23 #pragma GCC optimize("-fwhole-program")
24 #pragma GCC optimize("-freorder-blocks")
25 #pragma GCC optimize("-fschedule-insns")
26 #pragma GCC optimize("inline-functions")
27 #pragma GCC optimize("-ftree-tail-merge")
28 #pragma GCC optimize("-fschedule-insns2")
29 #pragma GCC optimize("-fstrict-aliasing")
30 #pragma GCC optimize("-fstrict-overflow")
31 #pragma GCC optimize("-falign-functions")
32 #pragma GCC optimize("-fcse-skip-blocks")
33 #pragma GCC optimize("-fcse-follow-jumps")
34 #pragma GCC optimize("-fsched-interblock")
35 #pragma GCC optimize("-fpartial-inlining")
36 #pragma GCC optimize("no-stack-protector")
37 #pragma GCC optimize("-freorder-functions")
38 #pragma GCC optimize("-findirect-inlining")
39 #pragma GCC optimize("-fhoist-adjacent-loads")
40 #pragma GCC optimize("-frerun-cse-after-loop")
41 #pragma GCC optimize("inline-small-functions")
42 #pragma GCC optimize("-finline-small-functions")
43 #pragma GCC optimize("-ftree-switch-conversion")
44 #pragma GCC optimize("-foptimize-sibling-calls")
```

```
45 #pragma GCC optimize("-fexpensive-optimizations")
46 #pragma GCC optimize("-funsafe-loop-optimizations")
47 #pragma GCC optimize("inline-functions-called-once")
48 #pragma GCC optimize("-fdelete-null-pointer-checks")
```

交互题

输入由评测机输入，根据输入的内容输出询问，直到确定正确答案，这一类题目，往往限制你交流(或询问)的次数，让你猜出一个东西来，此类问题比较经典的技巧有二分答案、随机化

需要注意的是，如果输出了一些数据，这些数据可能被放置于内部缓存区里，而且或许没有被直接传输给interactor，出现Idleness limit exceeded。为了避免这种情况的发生，需要每次输出时用一种特殊的清除缓存操作。

```
1  fflush(stdout); //方式一
2  cout << flush; //方式二
3  //endl换行时也会清除缓存，但'\n'不会
4  //最好不要使用快读、关同步流等
```

```
1  //https://codeforces.com/contest/1999/problem/G1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  void sol() {
6      int l = 2, r = 1000;
7      while (l < r) {
8          int mid = l + r >> 1;
9          cout << "? l " << mid << endl;
10         int res; cin >> res;
11         if (mid != res) r = mid;
12         else l = mid + 1;
13     }
14     cout << "! " << l << endl;
15 }
16
17 int main() {
18     int T = 1;
19     cin >> T;
20     while (T--) {
21         sol();
22     }
23 }
```

一些语法/标准

重载运算符

一些可重载运算符的列举

- 一元运算：`+`（正号）；`-`（负号）；`~`（按位取反）；`++`；`--`；`!`（逻辑非）；`*`（取指针对应值）；`&`（取地址）；`->`（类成员访问运算符）等。
- 二元运算：`+`；`-`；`&`（按位与）；`[]`（取下标）；`=`（赋值）；`>` `<` `>=` `<=` `==` `+=` `&=` 等。
- 其它：`()`（函数调用）；`""`（后缀标识符，C++11 起）；`new`（内存分配）；`,`（逗号运算符）；`<=>`（三路比较(C++20 起) 等。

实现

重载运算符分为两种情况，重载为成员函数或非成员函数。

当重载为成员函数时，因为隐含一个指向当前成员的 `this` 指针作为参数，此时函数的参数个数与运算操作数相比少一个。

而当重载为非成员函数时，函数的参数个数与运算操作数相同。

其基本格式为（假设需要被重载的运算符为 `@`）：

```
1 class Example {
2     // 成员函数的例子
3     返回类型 operator @ (除本身外的参数) { /* ... */ }
4 };
5
6 // 非成员函数的例子
7 返回类型 operator @ (所有参与运算的参数) { /* ... */ }
```

```
1 #include <iostream>
2 #include <queue>
3 #include <set>
4 using namespace std;
5 const int P = 1e9 + 7;
6
7 struct Test {
8     int k;
9 };
10
11 struct Edge {
12     int a, b, w;
13     bool operator < (const Edge& e2) const { //set、map、pq等重载需要加const
14         if (a != e2.a) return a < e2.a;
15         if (b != e2.b) return b < e2.b;
```

```

16         return w < e2.w;
17     }
18     bool operator > (Edge& e2) {
19         if (a != e2.a) return a > e2.a;
20         if (b != e2.b) return b > e2.b;
21         return w > e2.w;
22     }
23     long long operator * (Edge& e2) {
24         return a * e2.a + b * e2.b + w * e2.w;
25     }
26     Edge operator += (Edge& e2){
27         a += e2.a; b += e2.b; w += e2.w;
28         return *this; //隐含了一个指向当前成员的this指针
29     }
30     long long operator * (Test& t) { //参数可以为其它类
31         return a * t.k + b * t.k + w * t.k;
32     }
33 };
34
35 Edge operator += (Edge& e, Test& t) { //非成员函数重载
36     return /*Edge*/{e.a - t.k, e.b - t.k, e.w - t.k};
37 }
38
39 struct cmp{
40     bool operator () (auto &e1, auto &e2) const { //cmp函数调用重载
41         return e1.a > e2.b; //pq小根堆, 堆顶为最小值
42         return e1.a < e2.b; //pq大根堆, 堆顶为最大值
43     }
44 };
45
46 int main() {
47     int n = 10; //cin >> n;
48     multiset<Edge> se;
49     priority_queue<Edge> pq; //优先队列默认重载<
50     //priority_queue<Edge, vector<Edge>, cmp> pq; //cmp函数调用重载
51     for (int i = 1; i <= n; i++) {
52         int x = abs((P * i * i) % 11) + 1;
53         Edge a = { x, x * i % 10, x * i * i % 10 };
54         se.insert(a);
55         pq.push(a);
56     }
57
58     se.insert({ 9, 4, 4 });
59     se.erase({ 9, 4, 4 });
60     cout << "multiset:" << endl;
61     for (const Edge& x : se) { cout << x.a << ' ' << x.b << ' ' << x.w << endl;
62 }
63
64     cout << "\npq:" << endl;
65     while (pq.size()) {
66         Edge x = pq.top();
67         cout << x.a << ' ' << x.b << ' ' << x.w << endl;
68         pq.pop();

```

```

67     }
68     cout << "\ne1 = {1,2,3}, e2 = {4,5,6}\n";
69     cout << "e1 * e2 = ";
70     Edge e1 = { 1,2,3 }, e2 = { 4,5,6 };
71     cout << e1 * e2 << endl;
72
73     cout << "e1 += e2 => ";
74     e1 += e2;
75     cout << e1.a << ' ' << e1.b << ' ' << e1.w << endl;
76     Test t = { 10 };
77     cout << "e1 * t = ";
78     cout << e1 * t << endl;
79     cout << "e1 = e1-t => ";
80     e1 = e1 - t;
81     cout << e1.a << ' ' << e1.b << ' ' << e1.w << endl;
82     return 0;
83 }

```

Lambda表达式

```
1 | [capture list] (parameter list) -> return type { function body }
```

- `capture list` 是捕获列表，用于指定 Lambda表达式可以访问的外部变量，以及是按值还是按引用的方式访问。捕获列表可以为空，表示不访问任何外部变量，也可以使用默认捕获模式 `&` 或 `=` 来表示按引用或按值捕获所有外部变量，还可以混合使用具体的变量名和默认捕获模式来指定不同的捕获方式。
- `parameter list` 是参数列表，用于表示 Lambda表达式的参数，**可以为空**，表示没有参数，也可以和普通函数一样指定参数的类型和名称，还可以在 c++14 中使用 `auto` 关键字来实现泛型参数。
- `return type` 是返回值类型，用于指定 Lambda表达式的返回值类型，**可以省略**，表示由编译器根据函数体推导，也可以使用 `->` 符号显式指定，还可以在 c++14 中使用 `auto` 关键字来实现泛型返回值。
- `function body` 是函数体，用于表示 Lambda表达式的具体逻辑，可以是一条语句，也可以是多条语句，还可以在 c++14 中使用 `constexpr` 来实现编译期计算。

```

1 | int n,x;cin >> n >> x;
2 | for(int i = 1;i <= n;i++){
3 |     cin >> a[i];
4 | }
5 |
6 | auto check = [&](int mid){//使用例子
7 |     return a[mid] >= x;
8 | };

```

```

9
10 int l = 1, r = n;
11 while(l < r){
12     int mid = l + r >> 1;
13     if(check(mid)) r = mid;
14     else l = mid + 1;
15 }
16 cout << l << endl;

```

递归写法

```

1 //参数调用自己
2 auto fib = [&](auto &fib, int x){//递归求斐波那契
3     if(x == 1 || x == 2) return 1;
4     return fib(fib, x-1) + fib(fib, x-2);
5 };
6 cout << fib(fib, n) << endl;
7
8 auto dfs = [&](auto &&self, int u, int fa)->void{//递归调用出现在返回值之前,需要声明返回值类型
9     for(auto x:e[u]){
10         if(x == fa) continue;
11         self(self, x, u);
12     }
13 };

```

值捕获

值捕获的变量在 Lambda 表达式定义时就已经确定，不会随着外部变量的变化而变化。值捕获的变量默认不能在 Lambda 表达式中修改

```

1 int x = 10;
2 auto f = [x](auto a) -> int{ return a*x; };
3 x = 1;
4 cout << f(3) << endl;//结果为30, 不随x值的变化而改变

```

引用捕获

在捕获列表中使用 & 加变量名，表示将该变量的引用传递到 Lambda 表达式中，作为一个数据成员。引用捕获的变量在 Lambda 表达式调用时才确定，会随着外部变量的变化而变化。引用捕获的变量可以在 Lambda 表达式中被修改

```

1  int x = 10;
2  auto f = [&x](auto a) -> int{ x = 100;return a*x; };
3  cout << f(3) << endl; //结果为300，随x值的变化而改变
4  cout << x << endl; //x=100

```

隐式捕获

在捕获列表中使用 `=` 或 `&`，表示按值或按引用捕获 Lambda 表达式中使用的所有外部变量。这种方式可以简化捕获列表的书写，避免过长或遗漏。隐式捕获可以和显式捕获混合使用，但不能和同类型的显式捕获一起使用

```

1  int x = 10;
2  int y = 20;
3  auto f = [=, &y] (int z) -> int { return x + y + z; }; // 隐式按值捕获 x，显式按引用捕获 y
4  x = 30;
5  y = 40;
6  cout << f(5) << endl; //输出55，不受外部 x 的影响，受外部 y 的影响

```

初始化捕获 (init capture)：C++14 引入的一种新的捕获方式，它允许在捕获列表中使用初始化表达式，从而在捕获列表中创建并初始化一个新的变量，而不是捕获一个已存在的变量。这种方式可以使用 `auto` 关键字来推导类型，也可以显式指定类型。这种方式可以用来捕获只移动的变量，或者捕获 `this` 指针的值。

```

1  int x = 10;
2  auto f = [z = x + 5] (int y) -> int { return z + y; }; // 初始化捕获 z，相当于值捕获 x + 5
3  x = 20; // 修改外部的 x
4  cout << f(5) << endl; // 输出 20，不受外部 x 的影响

```

使用例子

作为函数参数，自定义排序准则

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  struct Edge{
6      int a,b;
7  }e[100005];
8
9  int main(){
10     int n;cin >> n;
11     for_each(e+1,e+n+1, [](auto &x){cin >> x.a >> x.b;}); //Edge可换用auto

```

```

12     sort(e+1,e+n+1,[](auto &x,auto &y){if(x.a != y.a) return x.a > y.a;else
    return x.b > y.b;});
13     for_each(e+1,e+n+1,[](auto &x){cout << x.a << ' ' << x.b << endl;});
14
15     return 0;
16 }

```

```

1 int a,b,c; cin >> a >> b >> c;
2 auto add = [&]() {w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;};
3 add();

```

预编译头文件

通过预编译头文件<bits/stdc++.h>，加快编译万能头的时间，以windows下的gcc14.2.0编译器为例

```

1 #进入头文件所在目录 例如 C:\mingw64\include\c++\14.2.0\x86_64-w64-mingw32\bits
2 #打开cmd输入以下指令 其中-x c++-header可以告诉编译器这是头文件,确保编译器正确处理文件
3 g++ stdc++.h -O2 -x c++-header -o stdc++.h.gch
4 #进入源cpp文件所在目录
5 #预编译头文件的编译器版本、编译选项(如-std=c++23、-O2等)必须与后续编译完全一致。
6 g++ 1.cpp -O2 -o -1.exe

```

gcc标准

__int128

__int128 仅仅是 GCC 编译器内的东西，不在 C++标准内，且仅 **GCC4.6** 以上**64位**版本支持

数据范围

__int128 数据范围： $-2^{127} \sim 2^{127} - 1$ 大于 $1.7e38$

unsigned __int128 数据范围： $0 \sim 2^{128}$

输入输出

由于不在 C++ 标准内，没有配套的 `printf` `scanf` `cin` `cout` 等输入输出，只能手写，请不要与 `ios::sync_with_stdio(false)` 同时使用，否则会造成输入输出混乱。

使用方法

与int、long long 等整型类似，支持各种运算，如+ - * / %、移位<< >>、& | ! ^、== >
< 等多种操作，可以直接与int进行运算

初始化

可以使用整型范围内的值直接赋初值 如 `__int128 a = 114514;`

可以使用浮点数赋值, 但超出浮点数有效精度的部分赋值不准确,

`__int128 a = 1e30` 结果为 100000000000000000019884624838656

`__int128 a = 1e40` 若超出 `__int128` 数据范围, 则会赋值为最大值, 即 $2^{127} - 1$

如果要自定义赋值为一个很大的数, 可以用 `__int128 a = (__int128(1) << 126)` 的方式
也可以写一个函数, 将字符串转为 `__int128`

```
1 __int128 sto__int128(const string &s){
2     __int128 x = 0, f = 1;
3     for(int i = 0; i < s.size(); i++){
4         if(s[i] == '-') { f = -1; }
5         else { x *= 10; x += s[i] - '0'; }
6     }
7     return x*f;
8 }
9 __int128 n = sto__int128("-114514191981000");
```

```
1 //使用例题 https://vjudge.net/contest/233969#problem/I
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 inline __int128 read(){
6     __int128 x = 0, f = 1;
7     char ch = getchar();
8     while(ch < '0' || ch > '9'){
9         if(ch == '-') f = -1;
10        ch = getchar();
11    }
12    while(ch >= '0' && ch <= '9'){
13        x = x*10 + ch - '0';
14        ch = getchar();
15    }
16    return x*f;
17 }
18
19 inline void write(__int128 x){
20     if(x < 0) { putchar('-'), x *= -1; }
21     if(x > 9) write(x/10);
22     putchar(x % 10 + '0');
23 }
24
25 void soviet(){
26     vector<__int128> a(4);
27     __int128 ans = 0;
28     for(int i = 0; i < 4; i++){
29         a[i] = read();
30         ans += a[i];
31     }
32     write(ans);
```

```
33     cout << '\n';
34 }
35
36 int main() {
37     int M_T = 1; //std::ios::sync_with_stdio(false), std::cin.tie(0);
38     std::cin >> M_T;
39     while(M_T--){ soviet(); }
40     return 0;
41 }
```

__cplusplus

```
1 | cout << __cplusplus << endl; // 可以直接输出预处理器宏参数
```

不同参数对应的默认c++标准	
1	C++ pre-C++98
199711L	C++98
201103L	C++11
201402L	C++14
201703L	C++17
202002L	C++20
202302L	C++23

编译时如果存在类似 `-std=c++17` 的选项，则显式指定了标准；如果未指定，则使用 `g++` 默认的标准（取决于版本）。

不同版本的 `g++` 默认使用不同的 C++ 标准：使用bash指令 `gcc --version` 可以查看 `g++` 版本

[所有Gcc版本对C和C++的支持情况（超详细版本）-CSDN博客](#)

文件读写

```

1 //正式提交时请务必注释掉
2 int main() {
3     freopen("in.txt","r",stdin);
4     // freopen("out.txt","w",stdout);
5     int a,b; cin >> a >> b;
6     cout << a + b << endl;
7 }

```

```

1 #windows cmd控制台读写方式
2 a.exe < in.txt > out.txt
3
4 #linux 控制台
5 ./a.out < in.txt > out.txt

```

可变模版参数

支持传入任意类型，任意数量的参数，可以用于Debug。

```

1 template<class... Args>
2 void f(Args... args) {
3     ((cout << args << ' '), ...);
4     // for (const auto& arg : {args...}) {
5     //     cout << arg << ' ';
6     // }
7 }

```

时间复杂度

仅供参考

数据范围		
10	$O(2^n), n!$	dfs+剪枝、状压dp、全排列、
100	$O(n^3)$	floyd、区间dp、高斯消元、矩阵、网络流、
1000	$O(n^2) \sim O(n^2 \log n)$	dp、二分，朴素版Dijkstra、Bellman-Ford、二分图、
1e4	$O(n\sqrt{n})$	块状链表、分块、莫队、
1e5	$O(n \log n)$	线段树、树状数组、set/map、heap、拓扑排序、dijkstra、prim+heap、Kruskal、spfa、凸包、半平面交、二分、CDQ分治、整体二分、后缀数组、树链剖分、动态树、平衡树、

数据范围		
1e6	$O(n) \sim O(n \log n)$	单调队列、hash、双指针扫描、并查集、KMP、AC自动机、树状数组、heap、Dijkstra、Tarjan、
1e7	$O(n)$	双指针、kmp、AC自动机、线性筛、
1e9~1e12	$O(\sqrt{n}) \sim O(n^{\frac{2}{3}})$	判断质数、求欧拉函数、数论分块、杜教筛、Min25筛、
1e18	$O(\log n)$	最大公约数、快速幂、数位DP、二分、
1e1000	$O(\log^2 n)$	高精度加减乘除、
1e100000	$O(\log k * \log \log k)$	高精度加减、FFT/NTT、

```

1 // (n+n/2+n/3+...+n/n) = n log n 调和级数
2 // https://codeforces.com/contest/1996/problem/D
3 // ab+bc+ac <= n, a+b+c <= x, 0 < a,b,c
4 for(int i = 1; i <= n; i++){ //枚举所有a
5     for(int j = 1; i*j <= n; j++){ //枚举所有b
6         // 则 0 < c <= min((b-ab)/(a+b), x-(a+b))
7     }
8 }

```

```

1 // https://codeforces.com/contest/1991/problem/F
2 1 1 2 3 5 8 ...
3 fib(26) 约 1.2e5
4 fib(45) 约 1.1e9
5 // 若 a[i] <= 1e9, 则长度 >= 45 的序列一定能构成一个三角形

```

```

1 // 阶乘
2 8! 约 4.0e4
3 12! 约 4.7e8

```

```

1 // 质数个数
2 1e8 约 5.7e6 个
3 1e7 约 6.6e5 个
4 1e6 约 7.8e4 个
5 1e5 约 9.5e3 个

```

```

1 // 最多约数个数
2 1e5(83160) 128 个
3 1e9(735134400) 1344 个
4 1e18(897612484786617600) 103680 个

```

$$\sum_{i=1}^n \sum_{j=1}^n (a_i * a_j) \ [i \neq j] = \sum_{i=1}^n (2 * a_i * \text{sum}_{i-1})$$

$$\sum_{i=1}^n \sum_{j=i+1}^n (a_i * a_j) = (sum^2 - \sum_{i=1}^n a_i^2) / 2$$

$$\sum_{i=1}^n \sum_{j=1}^n (a_i + a_j) = sum * n * 2, \text{ 若有 } k \text{ 层 } \sum \text{ 求和, 则答案为 } sum * n^{k-1} * k$$

$$\sum_{i=1}^n \sum_{j=i+1}^n (a_i + a_j) = sum * (n - 1)$$

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor^2 \varphi(d), \text{ } \varphi() \text{ 为欧拉函数, 可以用整除分块进一步优化为 } O(\sqrt{n})$$

区间 $[l, r]$ 内经过点 i 的所有子区间长度之和:

$$S = \sum_{a=l}^i \sum_{b=i}^r (b - a + 1) = \frac{(i-l+1)*(r-i+1)*(r-l+2)}{2}$$

```

1 //1~n内数位非递减的数,如123,223等
2 //https://atcoder.jp/contests/typical90/tasks/typical90_y
3 //n = 1e18,cnt = 5e6
4 //n = 1e9,cnt = 5e4
5 #include <iostream>
6 using namespace std;
7 long long n,cnt = 0;
8 void dfs(long long u){
9     if(u > n) return;
10    cnt++;
11    for(int i = u%10;i <= 9;i++){
12        dfs(u*10+i);
13    }
14 }
15
16 int main(){
17     cin >> n;
18     for(int i = 1;i <= 9;i++){ dfs(i); }
19     cout << cnt << endl;
20 }

```

```

1 //1~根号n枚举
2 for(int i = 1;i*i <= n;i++); //建议使用,大概3周期,需要注意数据范围
3 for(int i = 1;i <= sqrt(n);i++); //大概10周期(需要开启O2优化或循环外提前计算sqrt(n)+eps)
4 for(int i = 1;i <= n/i;i++); //大概40周期,除法运算较慢,如果时间紧不建议使用

```

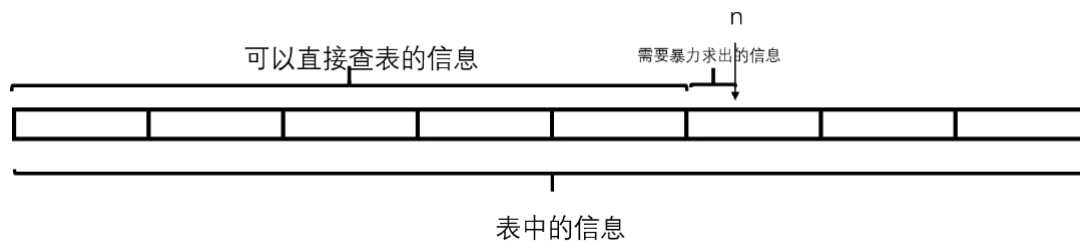
握手定理

n 个人握手，每人握手 m 次，握手总次数 $S = \frac{nm}{2}$

给定无向图 $G=(V,E)$ ，有 $\sum \deg(V) = 2E$ ，图中所有结点的度数之和等于边数的两倍

- 1 //https://codeforces.com/gym/104337/problem/H
- 2 //a1+a2+a3+... = n 不同的a最多约为 $\sqrt{2n}$ 种

分块打表



[Harmonic Number - LightOJ 1234 - Virtual Judge \(vjudge.net\)](https://vjudge.net/contest/428194#problem/H)

求 $\sum_{i=1}^n \frac{1}{i}$, $1 \leq n \leq 10^8$, 最多 10^4 组测试样例。

如果直接用前缀和预处理，数组需要开到 10^8 但空间不允许

我们可以每 $len = 1000$ 块为一组，每一块预处理。查询时，对于在表中的整块部分直接计算，其它部分暴力枚举

```
1 //O(N)预处理,O(len)查询,其中len为每块的大小
2 #include <iostream>
3
4 const int N = 100005;
5 double s[N]; //统计的为块的前缀和信息
6 int len = 1000;
7
8 void init(int n){
9     double ans = 0;
10    for(int i = 1; i <= n*len; i++){
11        ans += 1.0/i;
12        if(i%len == 0){
13            s[i/len] += ans;
14        }
15    }
16 }
17
18 void sol(){
19     int n; std::cin >> n;
20     int k = n/len;
21     double ans = s[k];
22     for(int i = k*len+1; i <= n; i++){
```

```

23     ans += 1.0/i;
24 }
25 printf("%.10lf\n",ans);
26 }
27
28 int main(){
29     init(N-1);
30     int t; std::cin >> t;
31     for(int i = 1;i <= t;i++){
32         printf("Case %d: ",i);
33         sol();
34     }
35 }

```

随机数生成

```

1 //mt19937
2 #include <iostream>
3 #include <random>
4 #include <chrono>
5 using namespace std;
6
7 auto SEED = chrono::steady_clock::now().time_since_epoch().count();//利用时间生成
    随机种子
8 mt19937_64 rng(SEED); //返回值为ull,注意转有符号整型时可能为负数
9
10 template<typename T> //随机返回一个区间[l,r]内的数
11 T rnd(T &l, T &r) { return rng() % (r - l + 1) + l; }
12
13 int main() {
14     cout << rng() << endl;//每次调用rng()时,生成的数都不同
15     //cout << mt19937_64(1 + SEED)() << endl; //生成种子为1下的固定随机数
16 }

```

```

1 //xor_shift
2 //异或和移位每次都在上一次产生的值上产生新的值,因为在很大的值中舍弃了一些值,所以每次产生的值看起来就象是随机值,也就是多次shift下的伪随机数,常用于树哈希
3 //这种哈希十分好些且比mt19937_64(seed)()快,但随机性不强
4 const unsigned long long mask = mt19937_64(time(0))(); //随机一个数作为固定种子
5 unsigned long long shift(unsigned long long x){ //x在该种子下得到的随机数
6     x ^= mask;
7     x ^= x << 13;
8     x ^= x >> 7;
9     x ^= x << 17;
10    x ^= mask;
11    return x;
12 }

```

```

1 //更强的哈希函数
2 const auto RAND = std::chrono::steady_clock::now().time_since_epoch().count();
3 static long long splitmix64(long long x) {
4     x += 0x9e3779b97f4a7c15;
5     x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
6     x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
7     return (x ^ (x >> 31)) + RAND;
8 }

```

```

1 //打乱数组 c++11起
2 shuffle(v.begin(), v.end(), rng); // 需要配合随机数生成器使用

```

对拍

实时快速生成随机样例，用于调试程序。

写一个对拍程序 `dp.exe`，不断调用数据生成器 `data.exe` 生成随机数据，通过将自己的程序 `sol.exe` 和标准程序（或保证正确的暴力解）`std.exe` 运行结果进行比对，一旦结果不一致立即终止程序。利用问题样例调试程序。

```

1 //dp.cpp windows
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int t = 0;
7     string sol = "sol";
8     cout << "filename(without .exe):";
9     cin >> sol;
10

```

```

11     while (1) {
12         system("data.exe > data.in");
13         system("std.exe < data.in > std.out");
14         system((sol + ".exe < data.in > sol.out").c_str());
15
16         if (system("fc std.out sol.out > nul")) {
17             cout << "\nWA on test " << ++t << "\n\nInput:\n";
18             system("type data.in");
19             cout << "\nYour output:\n";
20             system("type sol.out");
21             cout << "\nCorrect output:\n";
22             system("type std.out");
23             system("pause");
24             break;
25         }
26         cout << "Test " << ++t << " OK" << endl;
27     }
28 }

```

```

1 //dp.cpp  linux
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int t = 0;
7     string sol;
8     cout << "filename(without .out):";
9     cin >> sol;
10
11     while (1) {
12         system("./data > data.in");
13         system("./std < data.in > std.txt");
14         system(("./" + sol + " < data.in > sol.txt").c_str());
15
16         if (system("diff -q std.txt sol.txt > /dev/null")) {
17             cout << "\nWA on test " << ++t << "\n\nInput:\n";
18             system("cat data.in");
19             cout << "\nYour output:\n";
20             system("cat sol.txt");
21             cout << "\nCorrect output:\n";
22             system("cat std.txt");
23             cin.ignore();
24             break;
25         }
26         cout << "Test " << ++t << " OK" << endl;
27     }
28 }

```

```

1 //data.cpp
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 auto SEED = std::chrono::steady_clock::now().time_since_epoch().count();
6 std::mt19937_64 rng(SEED);
7
8 template<typename T> //rnd(l, r)生成区间[l,r]内的一个随机数
9 T rnd(const T &l, const T &r) { return rng() % (r - l + 1) + l; }
10
11 std::vector<int> random_permutation(int n) { //生成[1~n]的一个乱序排列
12     std::vector<int> a(n);
13     std::iota(a.begin(), a.end(), 1);
14     std::shuffle(a.begin(), a.end(), rng);
15     return a;
16 }
17
18 int main() { //例如生成数组a[n]
19     int n = rnd(1, 100000);
20     cout << n << '\n';
21     while (n--) {
22         cout << rnd<int>(1, 1e9) << ' ';
23     }
24 }

```

```

1 //dp.cpp windows(太长,仅日常使用)
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int t = 0;
7     std::cout << "sol filename:";
8     std::string sol = "1.cpp"; std::cin >> sol;
9     std::cout << std::endl;
10
11     int suf_pos = sol.find_last_of(std::string("."));
12     std::string suf = ".cpp";
13     if (suf_pos != -1) {
14         suf = sol.substr(suf_pos);
15     }
16
17     if (sol.size() >= 4 && sol.substr(sol.size() - 4) == ".cpp") {
18         sol.pop_back(); sol.pop_back(); sol.pop_back(); sol.pop_back();
19     }
20
21     while (1) {
22
23         system("data.exe > data.in");
24

```

```

25     std::cout << "Test:" << std::left << std::setw(4) << ++t;
26
27     system("std.exe < data.in > std.out");
28
29     std::cout << "->";
30
31     clock_t c1 = clock();
32     if (suf.empty() || suf == ".exe" || suf == ".cpp") system((sol + " <
data.in > sol.out").c_str());
33     else if (suf == ".py") system(("python " + sol + " <data.in >
sol.out").c_str());
34     clock_t c2 = clock();
35
36     std::cout << std::right << " " << std::setw(4) << c2 - c1 << "ms ";
37
38     if (system("fc std.out sol.out > diff.log")) { // linux下为diff命令对比文
件
39         system("cls");
40         printf("\a");
41
42         std::cout << "WA on Test " << t << ":" << std::endl;
43         system("type data.in");
44         std::cout << std::endl;
45
46         std::cout << "Your Answer:" << std::endl;
47         system("type sol.out");
48         std::cout << std::endl;
49
50         std::cout << "Std Answer:" << std::endl;
51         system("type std.out");
52         std::cout << std::endl;
53
54         system("pause");
55         break;
56     }
57     std::cout << "OK ";
58     std::cout << std::endl;
59 }
60
61 return 0;
62 }

```

外部链接

一些好van的网站

CLIST: <https://clist.by/> : 查看近期有哪些网络比赛, 多达上百个OJ, 支持自定义列表, 查看个人比赛统计

Board-XCPCIO: <https://board.xcpcio.com/> : XCPC比赛榜单、外榜

QOJ: <https://qoj.ac/contests> : XCPC补题、vp

[OEIS: https://oeis.org/](https://oeis.org/) : OEIS整数数列线上大全、用于数列打表找规律

[CPC Finder: https://cpcfinder.com/](https://cpcfinder.com/) : 竞赛选手成绩查询、开盒

[Graph Editor: https://csacademy.com/app/graph_editor/](https://csacademy.com/app/graph_editor/) : 简易的图论作图工具

[ACMer.info: https://acmer.info/](https://acmer.info/) : 整理和分享算法竞赛相关的群组、博客、比赛平台等资源

[AlgoWiki: https://www.algowiki.cn/](https://www.algowiki.cn/): 竞赛wiki信息平台

[OI Wiki: https://oi-wiki.org/](https://oi-wiki.org/) : 编程竞赛知识整合站点

[LaTeX公式编辑器: https://www.latexlive.com/](https://www.latexlive.com/): 在线LaTeX公式编辑器

[cppreference: https://en.cppreference.com/](https://en.cppreference.com/) : c++参考手册

.vimrc

个人vim简易配置

```
1 | imap jk <Esc>
2 | nmap <space> :
3 |
4 | set cin
5 | set sw=4
6 | set ts=4
7 |
8 | map <F5> :w <cr> :!clear & g++ % -o %:r<cr>
9 | map <F6> :!clear & ./%:r <cr>
10 | map <C-F6> :!clear & ./%:r < in <cr>
11 | "map <C-F6> :!clear & time ./%:r.out < in \| tee out <cr>
12 | "set mouse=a
13 | "syn on
```

OI
